

Template and CPA Side Channel Attacks on the Kyber/ML-KEM Pair-Pointwise Multiplication

Sedric Nkotto

SRC Security Research & Consulting GmbH, Bonn, Germany

sedric.nkotto@src-gmbh.de

Abstract.

Kyber a.k.a ML-KEM is a quantum-safe Key Encapsulation Mechanism, that has been standardized by NIST under FIPS-203 and will definitely in the coming years be implemented in several commercial products. Following for instance this report <https://radar.cloudflare.com/adoption-and-usage?dateRange=52w>, one can notice that since the end of 2024, the post-quantum encrypted share of the HTTPS request traffics has been increasing and is about to reach 50% now. However, the resilience of implementations against side channel attacks is still an open and practical concern. One of the drawbacks of the ongoing side channel analysis research related to PQC schemes is the availability of open-source datasets. Luckily some open-source datasets start popping up. For instance, the one recently published by Rezaeezade *et al.* in the paper 2025/811. This dataset captures power consumption during a pair-pointwise multiplication occurring in the course of ML-KEM decapsulation process and involving the decapsulation (sub)key and ciphertexts. In this paper we present both a profiled (template) and an unprofiled (CPA) side channel attacks targeting that pair-pointwise multiplication, which yield, in both cases, a complete recovery of the decapsulation secret (sub)key.

Keywords: Template Side-Channel Attack, LDA, Correlation Power Attack, Post Quantum Cryptography, Kyber, ML-KEM

Contents

1	ML-KEM	2
1.1	High level description of ML-KEM	2
1.2	Polynomial multiplication in ML-KEM	3
2	Profiled SCA Attack	3
2.1	Dataset	4
2.2	Targets	4
2.3	Template Attack	5
3	Unprofiled SCA Attack	8
3.1	Correlation-based Leakage detection	8
3.2	Correlation Power Attack	9
4	Conclusion	11

Introduction

The NIST initiated in 2016 a standardization program to select algorithms resilient against attackers possessing a cryptographically relevant quantum computer. The program delivered the first standards in 2024, which encompass among others FIPS-203 (ML-KEM). This is the first step towards deployment for public use. But before deploying a given implementation, it is crucial to guarantee its resilience against side channel attackers. This paper presents a profiled and an unprofiled SCA attack on the ML-KEM pair-pointwise multiplication. The latter represents the coefficient-wise multiplication of two polynomials in the NTT-domain and is implemented by the `PQCLEAN-MLKEM768-basemul` subroutine in the reference implementation of ML-KEM (cf. PQClean Github repository¹). Our attacks target the dataset provided in the recently published paper [3] by Rezaeezade *et al.* This dataset contains power measurements captured during a "pair-pointwise multiplication" in the decapsulation procedure of ML-KEM, running on an STM32F3 micro-controller at 7.372 MHz. For more details how the product of a pair of polynomial coefficients in the NTT domain are computed in ML-KEM, one can also consider the algorithm `BaseCaseMultiply` in [2, sec. 4.3.1, algorithm 12]. We also recall that the considered implementation is a pure C one and the dataset used an O3 level optimization. More information about the dataset can be found in [3].

1 ML-KEM

1.1 High level description of ML-KEM

ML-KEM is a quantum-safe lattice-based Key Encapsulation Mechanism standardized in 2024 by the US National Institute of Standards and Technology (NIST) under FIPS-203. ML-KEM encompasses three main algorithms namely,

1. **KeyGen: Key Generation**, which is used by one of the parties (we call **Amadou**) to produce a public key (**pk**) and a secret key (**sk**). The public key **pk** is subsequently sent to the other party (we call **Bineta**).

Let us consider the quotient ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$, with $q = 3329$.

In order to produce **sk** and **pk**, Amadou pseudo-randomly samples a matrix $A \in R_q^{k \times k}$ and "small" vectors $s, e \in R_q^k$. The key pair (**sk**, **pk**) are defined as **sk** = s and **pk** = (A, t) with $t = A \cdot s + e(\text{mod } q)$.

2. **Encapsulation: Encaps**

Bineta uses Amadou's public key **pk**, to encapsulate a shared secret key m by encoding it into a polynomial in R_q and computing a ciphertext **c** from it. To do so, Bineta samples "small" random vectors $r, e_1 \in R_q^k$ and $e_2 \in R_q$ and computes the following:

$$u = A^T \cdot r + e_1(\text{mod } q) \text{ and } v = t^T \cdot r + e_2 + \text{Encode}(m) \cdot \left\lfloor \frac{q}{2} \right\rfloor (\text{mod } q)$$

She subsequently sends $\mathbf{c} = (u, v)$ to Amadou.

3. **Decapsulation: Decaps**

Amadou uses his secret key **sk** = s together with the ciphertext $\mathbf{c} = (u, v)$ received from Bineta, to produce the shared secret key m . He does it by computing

$$v' = v - u^T \cdot s(\text{mod } q)$$

¹<https://github.com/pq-crystals/kyber>

After decoding v' , Amadou recovers the shared secret m . “Decoding” here means, coefficients of v' close to zero are decoded to 0 and the ones close to $\lfloor \frac{q}{2} \rfloor$ are decoded to 1.

Remark: As mentioned before, this is a simplified description. In a more detailed one, the actual shared key is obtained from m by computing

$$K = \text{SHA3-512}(m \parallel \text{SHA3-256}(\mathbf{pk}))$$

Spoiler: In the subsequent sections, our side channel attacks will be targeting the above-highlighted intermediate value.

1.2 Polynomial multiplication in ML-KEM

Let us recall that all scalar products of vectors in R_q^k (e.g. $u^T \cdot s$) involve polynomials multiplications in R_q . These are among the most expensive operations in ML-KEM. To speed them up, a technique called Number Theoretic Transform (NTT) is used. This is the modular arithmetic version of the Fast Fourier Transform (FFT).

Let $f = f_0 + f_1X + \dots + f_{255}X^{255}$ be a polynomial in R_q , which can also be represented as the tuple $(f_0, f_1, \dots, f_{255})$. The NTT-representation of f is given by

$$\text{NTT}(f) = (\widehat{f_{0,0}} + \widehat{f_{0,1}}X, \widehat{f_{1,0}} + \widehat{f_{1,1}}X, \dots, \widehat{f_{127,0}} + \widehat{f_{127,1}}X)$$

whereby $\widehat{f_{i,0}} + \widehat{f_{i,1}}X = f \bmod (X^2 - \zeta^{2\text{BitRev}_7(i)+1})$, with $\zeta = 17 \pmod{q}$ a primitive 256-th root of unity mod q and $\text{BitRev}_7(i)$ the bit reverse of the 7-bit representation of i .

The NTT-transform has an inverse transformation INTT and given two polynomials $f = f_0 + f_1X + \dots + f_{255}X^{255}$ and $g = g_0 + g_1X + \dots + g_{255}X^{255}$ in R_q , we have

$$f \times g = \text{INTT}(\text{NTT}(f) \circ \text{NTT}(g))$$

$\text{NTT}(f) \circ \text{NTT}(g)$ is a component-wise multiplication. That means the i -th component of this product is

$$(\widehat{f_{i,0}} + \widehat{f_{i,1}}X) \cdot (\widehat{g_{i,0}} + \widehat{g_{i,1}}X) \bmod (X^2 - \zeta^{2\text{BitRev}_7(i)+1})$$

which yields after reduction

$$\widehat{f_{i,0}} \cdot \widehat{g_{i,0}} + \widehat{f_{i,1}} \cdot \widehat{g_{i,1}} \zeta^{2\text{BitRev}_7(i)+1} + (\widehat{f_{i,0}} \cdot \widehat{g_{i,1}} + \widehat{f_{i,1}} \cdot \widehat{g_{i,0}}) X$$

This operation is performed in FIPS-203 by the `BaseCaseMultiply` subroutine (cf. [2, Algorithms 11 and 12]), also called `basemul`² in the reference implementation we will be considering in the next sections.

2 Profiled SCA Attack

We perform in this section a profiled template-based DPA attack on the above-mentioned ML-KEM Implementation Under Test (IUT) targeting the power traces dataset published in [3].

²<https://github.com/pq-crystals/kyber/blob/main/ref/ntt.c#L139-L146>

2.1 Dataset

The authors highlighted that the measurements in the dataset focused on the pairwise coefficient multiplication in the NTT domain implemented by the **basemul** function below (cf. [3, sec. 2, Listing 1.])

```

1 void basemul(int16_t r[2], const int16_t a[2], const int16_t b[2],
2             int16_t zeta)
3 {
4     1: r[0] = fqmul(a[1], b[1]);
5     2: r[0] = fqmul(r[0], zeta);
6     3: r[0] += fqmul(a[0], b[0]);
7     4: r[1] = fqmul(a[0], b[1]);
8     5: r[1] += fqmul(a[1], b[0]);
9 }

```

Listing 1: Kyber pair-pointwise multiplication in NTT Domain

We recall that $\text{fqmul}(a, b)$ outputs the product of a and b followed by a Montgomery reduction³.

Apart from the raw traces, the dataset provides

- the coefficients $a[0]$ and $a[1]$ of the secret key (fixed for all measurements),
- the coefficient $b[1]$ of the ciphertext for each measurement,
- and the corresponding outputs of the multiplications $\text{fqmul}(a[0], b[1])$ and $\text{fqmul}(a[1], b[1])$.

2.2 Targets

As we already announced in sec.1.1, our attack vector is $u^T \cdot s \pmod{q}$. More precisely, the component-wise multiplications of their NTT-representations.

We recall that u and s are vectors in R_q^k . i.e. the scalar product $u^T \cdot s \pmod{q}$ involves k polynomial multiplications. For each polynomial multiplication in the NTT-domain, 128 such multiplications are computed.

$$(a_0 + a_1 X) \cdot (b_0 + b_1 X) \pmod{X^2 - \zeta}$$

which after reduction yields

$$a_0 \cdot b_0 + a_1 \cdot b_1 \zeta + (a_0 \cdot b_1 + a_1 \cdot b_0) X \pmod{X^2 - \zeta}$$

That means, to recover the complete ML-KEM secret key, one should attack overall $128 \cdot k$ such operations, which are all realised by the **basemul** subroutine in our considered reference implementation. Throughout this paper, we only attack one of them because the available dataset only captures the power trace during a single one. Nevertheless, our attack can be replicated if the complete power trace is available. The advantage is also that, all the $128 \cdot k$ operations can be attacked in parallel. Therefore, as long as the attack complexity is concerned, there is no significant difference between attacking a single operation or all the $128 \cdot k$ ones.

We specifically target the secret sub-keys $a[0]$ and $a[1]$ in the operations

$$\text{fqmul}(a[0], b[1]) \text{ and } \text{fqmul}(a[1], b[1])$$

It is also worth mentioning that $a[0]$, $a[1]$, $b[0]$ and $b[1]$ are lying in the field $\mathbb{Z}_q$. This already provides information about the key-space of the secret sub-keys $a[0]$ and $a[1]$.

³<https://github.com/pq-crystals/kyber/blob/main/ref/ntt.c#L68-L69>

2.3 Template Attack

We recall that, the raw traces were compressed with a factor of 10, i.e. by taking the mean over sections of 10 sample points for each trace. The traces values were also rescaled to 16-bit unsigned integers. No further pre-processing of traces was applied.

2.3.1 Template-based leakage profiling

Our aim here is to recover the secret (sub)keys **a[0]** and **a[1]**.

The available traces were first of all split into two parts. 80% of the traces were used for profiling and 20% for matching/attack phase. The profiling traces were grouped into three classes depending on the Hamming weight of the MSByte of the outputs **fqmul(a[0], b[1])** and **fqmul(a[1], b[1])** as mentioned below:

Table 1: Traces shares

class	characteristic (<i>i=0 or 1</i>)
class0	HW(MSByte of fqmul(a[i], b[1])) = 0 or 3
class1	HW(MSByte of fqmul(a[i], b[1])) = 1
class2	HW(MSByte of fqmul(a[i], b[1])) = 2

The templates were per class constructed as the means of the belonging traces within each class.

2.3.2 Dimensionality reduction

Efficiently performing a side channel attack usually requires traces to be reduced, because of their large dimension. One should nevertheless make sure the most important features are preserved. We perform here a two-level dimensionality reduction.

- 1- First of all we consider our traces only at p points of interest (PoIs), whereby the PoIs are chosen w.r.t the highest SOSD values.
- 2- The second level of reduction is the linear discriminant analysis (LDA) based dimensionality reduction.

One could also perform only one of them. But we have observed in our case that using both of them yields a better outcome. One of the advantages of LDA is that, it does not only reduce the dimension of the initial data, but also projects the data in a smaller subspace, where the classes are better distinguishable. For the convenience of the reader, we provide in sec. 2.3.4 a high level description of linear discriminant analysis (LDA) tailored to our specific case.

2.3.3 SOSD

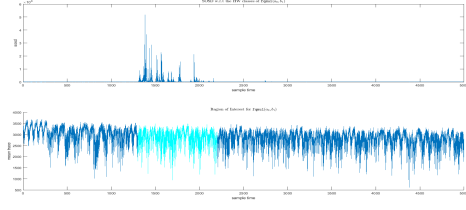
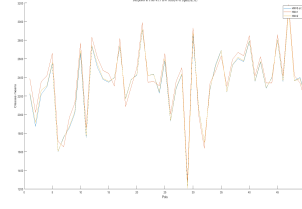
Let us first of all tackle the first level of dimensionality reduction. For the Points of Interest selection, the SOSD (Sum of Square Difference) was computed, which formula for the three classes is

$$(\mu_0 - \mu_1)^2 + (\mu_0 - \mu_2)^2 + (\mu_1 - \mu_2)^2,$$

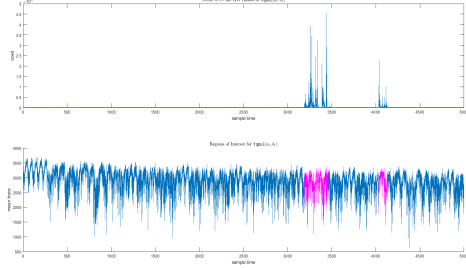
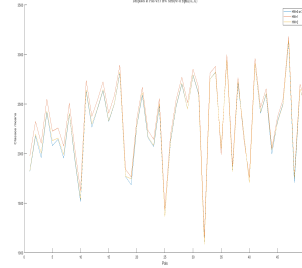
whereby μ_i represents the mean of the *class i*.

The figures below show the SOSD plots for the above-mentioned classes on the LHS and the corresponding means at PoIs on the RHS.

Clear spikes can be observed in the SOSD plots (cf. plottings on top in figures 1.1 and 2.1), which highlight the sample times at which the three classes are distinguishable. How good these classes are distinguishable is illustrated by figures 1.2 and 2.2, which show the mean traces of each of the three classes at the PoIs.

1.1: SOSD w.r.t. $\text{fqmul}(a[0], b[1])$ 

1.2: classes at PoIs w.r.t. SOSD values on LHS

2.1: SOSD w.r.t. $\text{fqmul}(a[1], b[1])$ 

2.2: classes at PoIs w.r.t. SOSD values on LHS

2.3.4 Linear Discriminant Analysis

Let us still consider our previous classes of traces *class0*, *class1* and *class2*. The aim of LDA is to find a linear projection of data to a subspace that minimizes the intra-class variance, while maximizing the inter-class variance. In other words find the matrix $\mathbf{V}$ that maximizes the ratio between inter-class scatter matrix and intra-class scatter matrix

$$\arg \max_{\mathbf{V}} \frac{|\mathbf{V}^T \mathbf{S}_B \mathbf{V}|}{|\mathbf{V}^T \mathbf{S}_W \mathbf{V}|}$$

with,

$$\mathbf{S}_B = \sum_{i=0}^2 n_i (\mu_i - \mu) \cdot (\mu_i - \mu)^T \in \mathbb{R}^{p \times p} \text{ and}$$

$$\mathbf{S}_W = \sum_{i=0}^2 \sum_{\mathbf{t} \in \text{class}_i} (\mathbf{t} - \mu_i) \cdot (\mathbf{t} - \mu_i)^T \in \mathbb{R}^{p \times p}$$

we recall that μ denotes the overall mean trace, μ_i the mean of traces in *class_i*, n_i the size of *class_i* and p the number of chosen PoIs as mentioned above in sec. 2.3.3.

The solution to that maximization problem is the matrix $\mathbf{V}$, which columns are made up of the eigenvectors, solutions of the generalized eigenvalues problem

$$(\mathbf{S}_B - \lambda_i \mathbf{S}_W) \mathbf{v}_i = 0 \quad (1)$$

In practice, only the $n_c - 1$ eigenvectors with the highest eigenvalues are considered, whereby n_c denotes the number of considered classes, i.e. $n_c = 3$ in our case.

We now proceed our template attack but with the projected traces data i.e.

$$\bar{\mathbf{t}} = \mathbf{V}^T \cdot \mathbf{t} \in \mathbb{R}^{n_c-1}$$

2.3.5 Key recovery by LDA-based template attack

A template here consists only of the mean trace (for each class). In other words, a multivariate Gaussian Template with covariance matrix the identity matrix. This is called in the literature reduced template analysis or the least square method (cf. [1, section 5.3]). The score of a given trace in the class i , is therefore computed by the formula

$$\text{score}(t, i) = (t - \mu_i) \cdot (t - \mu_i)^T \quad (2)$$

The score taking into account the projection by LDA is

$$\text{score}(t, i) = (\mathbf{V}^T \cdot t - \mathbf{V}^T \cdot \mu_i) \cdot (\mathbf{V}^T \cdot t - \mathbf{V}^T \cdot \mu_i)^T \quad (3)$$

Whereby t is the corresponding trace vector and μ_i the mean as mentioned before.

Let us for instance describe the attack procedure targeting the secret sub-key $\mathbf{a}[0]$.

As mentioned before, 80% of traces (i.e. 80.000 traces) are used as profiling traces and the remaining 20.000 traces are attack traces.

Procedure (profiling phase):

1. Share the profiling traces into three classes with respect to the Hamming weight of the MSByte of the `fqmul(a[0], b[1])` output (cf. table 1).
2. Compute the mean for each classes (μ_0, μ_1 and μ_2) at PoIs w.r.t the highest SOSD values.
3. Compute the inter-class scatter resp. intra-class scatter matrices $\mathbf{S}_B$ and $\mathbf{S}_W$.
4. Solve the corresponding generalized eigenvalues problem (cf. equation (1)) and generate the matrix $\mathbf{V}$, which column are the $n_c - 1 = 2$ eigenvectors with the highest eigenvalues.
Your profiling phase is done.

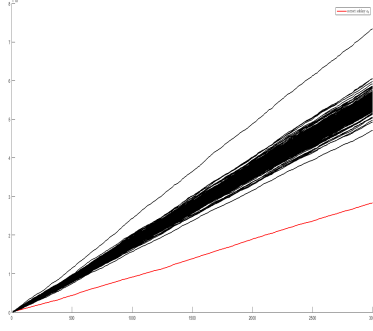
Procedure (matching phase):

1. Consider (part of) the remaining traces kept aside for the matching phase as mentioned above.
2. multiply each attack trace at PoIs $\mathbf{t}$ by $\mathbf{V}^T$ to get the projected traces. The subsequent part of the attack/matching phase is performed on the projected traces
3. To speed up your matching phase, a scores look-up table (LUT) can be precomputed. i.e. for each trace, compute the scores (see formula (3)) of that trace using each of the templates and store your scores LUT.
4. Run through the sub-key space. In the case of ML-KEM, $\mathbf{a}[0] \in \mathbb{Z}_q$, which means the sub-key space is $[0, q) = [0, 3329)$.
5. For each possible guess of the (ML-KEM decryption) sub-key $\mathbf{a}[0]$, sort the traces according to the Hamming weight of `MSByte(fqmul(guesses, b[1]))`. Compute subsequently the score of that guess by summing up the score of the traces accordingly. The previously pre-computed scores Lookup Table speeds up this step.
6. pick up the guess with the smallest score.

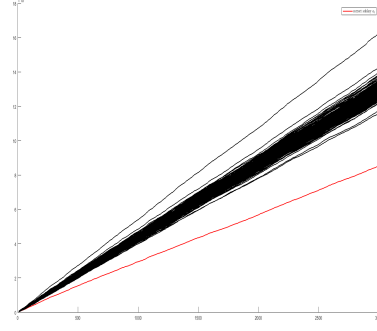
Remark: We recall that the PoIs mentioned in the procedure are selected by the highest SOSD values.

2.3.6 Results

As shown by the figures 3.1 and 3.2 below, the template attack yields the complete recovery of the ML-KEM decapsulation sub-keys. In both cases, the best guess, i.e. the one with the smallest score, matches the correct sub-key.

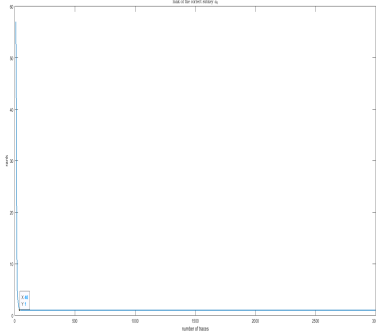


3.1: scores sub-key guesses for $a[0]$

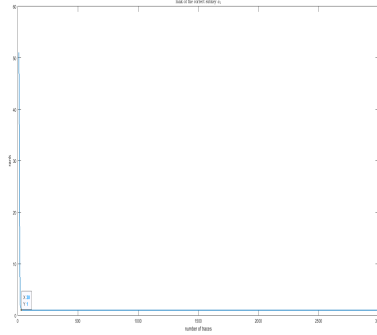


3.2: scores sub-key guesses for $a[1]$

We can notice in the figures 4.1 and 4.2 below that only 40 traces are necessary to recover $a[0]$ and even lower than that, 30 traces are enough to recover $a[1]$.



4.1: correct sub-key rank w.r.t number of attack traces guesses for $a[0]$



4.2: correct sub-key rank w.r.t number of attack traces for $a[1]$

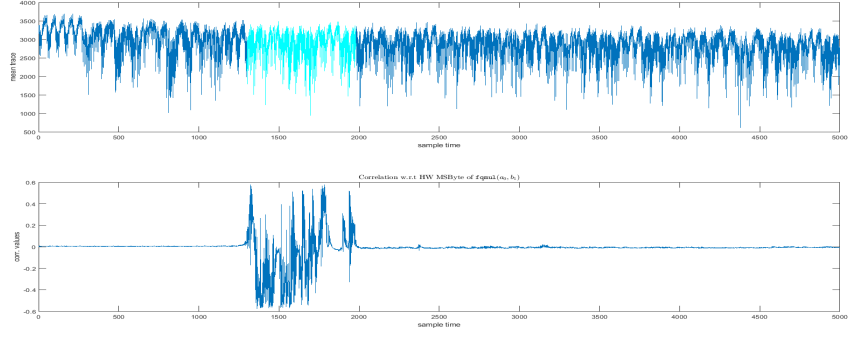
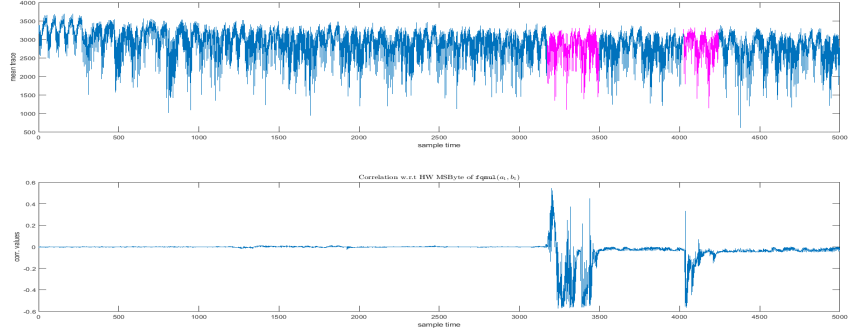
3 Unprofiled SCA Attack

3.1 Correlation-based Leakage detection

The traces were pre-processed the same way as before.

Figure 5 and figure 6 below show the correlation plots with respect to the Hamming weight of the MSByte of the intermediate values $f_{qmul}(a[0], b[1])$ resp. $f_{qmul}(a[1], b[1])$.

Note: we computed the intermediate values $f_{qmul}(a[0], b[1])$ and $f_{qmul}(a[1], b[1])$ on-the-fly while performing the attack and did not use the values stored in the dataset.

Figure 5: Correlation w.r.t. MSByte of $\text{fqmul}(a[0], b[1])$ Figure 6: Correlation w.r.t. MSByte of $\text{fqmul}(a[1], b[1])$

One clearly observes from the correlation plots that the above-mentioned intermediate values are susceptible to correlation analysis and the regions where the corresponding operations occur are easily identifiable. For the specific case of $\text{fqmul}(a[1], b[1])$, two distinct regions of interest can be seen. This is probably due to the fact that for the considered dataset, $b[0] = b[1]$, which makes $\text{fqmul}(a[1], b[1])$ to be computed twice (cf. Listing 1, line 1 and 5)

3.2 Correlation Power Attack

We have seen in sec. 3.1 that our considered intermediate values are very sensitive to correlation analysis. We therefore decided to also perform a CPA attack on the pair-pointwise multiplication in ML-KEM. The CPA attack investigates the correlation between the targeted intermediate values and the physical measurements (in this case power consumption). We recall (as mentioned in sec. 2.2) that our targeted intermediate values are $\text{fqmul}(a[0], b[1])$ when attacking $a[0]$ and $\text{fqmul}(a[1], b[1])$ when attacking $a[1]$. Let us describe the procedure for an attack targeting the secret sub-key $a[0]$. The procedure also applies mutatis mutandis to $a[1]$.

Procedure (CPA attack):

1. Run through the sub-key space (as mentioned before, $a[0] \in \mathbb{Z}_q$. i.e the sub-key space is $[0, q - 1]$)

2. For each possible guess $keyGuess$ of the (ML-KEM decryption) sub-key $\mathbf{a}[0]$, compute the hypothetical intermediate value $P(keyGuess)$, which is the Hamming weight of the most significant byte $\mathbf{MSByte}(\mathbf{fqmul}(\mathbf{keyGuess}, \mathbf{b}[1]))$.
3. Compute subsequently the score of that guess as the maximum absolute value of the Pearson correlation ρ between the traces and $P(keyGuess)$

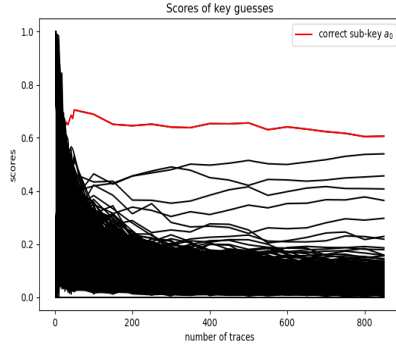
$$score(keyGuess) = \max_j (Abs(\rho(T_j, P(keyGuess))))$$

T_j denoting the trace vector at sample time j

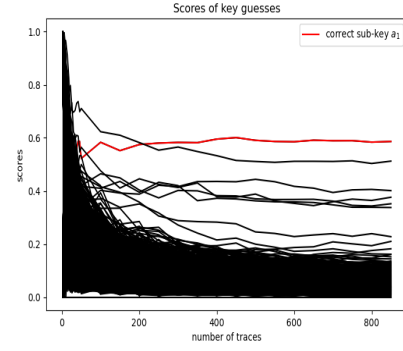
4. pick up the guess with the highest score.

Result CPA attack :

The figures below show the result of our CPA attack on the SCA dataset published in [3].

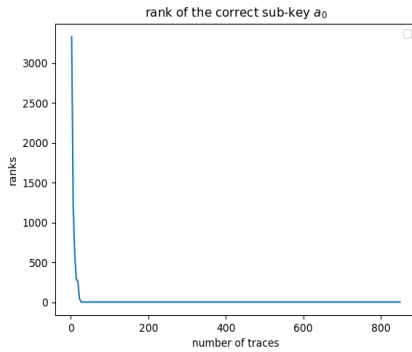


7.1: scores sub-key guesses for $\mathbf{a}[0]$

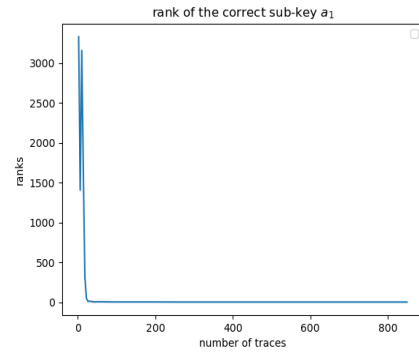


7.2: scores sub-key guesses for $\mathbf{a}[1]$

Remark: One observes in figures 8.1 and 8.2 that only 30 traces are enough to recover the sub-key $\mathbf{a}[0]$, whereas about 250 traces are required to recover $\mathbf{a}[1]$.



8.1: correct sub-key rank w.r.t number of attack traces guesses for $\mathbf{a}[0]$



8.2: correct sub-key rank w.r.t number of attack traces for $\mathbf{a}[1]$

4 Conclusion

While targeting the decapsulation subkeys $\mathbf{a}[0]$ and $\mathbf{a}[0]$, we performed both a profiled and unprofiled SCA attack on the dataset published in [3]. The dataset encompasses the recorded power consumption during a single pair-pointwise multiplication during the decapsulation procedure in ML-KEM. We recall that each of the considered pair-pointwise multiplications involves two coefficients of the NTT-representation of the secret key (cf. [2, sec. 4.3.1, algorithm 11]). We were able to recover the coefficients of a single pair-pointwise multiplication, out of a total of 256 coefficients. This is due to the fact that the available dataset only captures that single one. The same attack can be replicated in parallel over the 127 remaining pair-pointwise multiplications in case the corresponding traces are available.

Even though this investigation was not a comparative study between template attack and CPA attack, one notices for this dataset that the LDA-based template attack performs better, in the sense that, it succeeds with a much lower amount of traces compared to CPA attack.

References

- [1] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power analysis attacks: Revealing the secrets of smart cards*. Vol. 31. Springer Science & Business Media, 2008.
- [2] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication FIPS 203. U.S. Department of Commerce, 2024. URL: <https://doi.org/10.6028/NIST.FIPS.203>.
- [3] Azade Rezaeezade et al. “Side-Channel Power Trace Dataset for Kyber Pair-Pointwise Multiplication on Cortex-M4”. In: *Cryptology ePrint Archive* (2025).